

Course: Foundations of AI with Embeddings

Lecture 9: General-Purpose User Embeddings

AutoEncoder-coupled Transformer Network (AETN) · KDD 2020

Vladimir Kukushkin

Constructor University, 2026

Motivation & Problem

Why user embeddings from mobile app usage?

Mobile app usage encodes rich user interests:

- *Retention* — apps currently installed (long-term interest)
- *Installation* — recently added apps (emerging interests)
- *Uninstallation* — recently removed apps (changing interests)

Downstream applications that benefit:

- Feed recommendations
- Look-alike audience extension (advertising)
- Next app install prediction

Key insight

A user who just installed *Snapseed* and *Instagram* is likely interested in mobile photography — even if they never said so explicitly.

Traditional approach: handcrafted features per task, requires domain experts, sub-optimal across tasks.

Goal: learn *general-purpose* representations automatically.

Three core challenges

1. **Heterogeneous signals must be modeled collectively**

Retention, installation, and uninstallation are qualitatively different, yet they jointly describe user preferences. Encoding them separately and concatenating loses cross-signal interactions.

2. **Low-frequency & unevenly distributed over time**

Most users install/uninstall apps rarely; bursts can happen on any day. Traditional RNNs assume regular sequences and struggle with irregular time gaps.

3. **Long-tail app sparsity**

Popular apps (WeChat, Alipay) are on almost every phone — they are uninformative. Niche apps reveal true interests but may appear only a few hundred times in tens of millions of users. Standard embeddings fail on such rare items.

AETN Architecture

AETN: high-level overview

AutoEncoder-coupled Transformer Network — three parts trained jointly:

1. Retention Autoencoder

Learns *app embeddings* from co-occurrence in retention data.

Encodes the high-dimensional retention vector into a dense representation fed to the transformer.

2. Transformer Encoder

Takes retention encoding + installation + uninstallation sequences.

Compresses everything into a **bottleneck** — the user embedding $\tilde{e} \in \mathbb{R}^{128}$.

3. Transformer Decoder

Reconstructs installation and uninstallation sequences from the user embedding.

Forces the bottleneck to retain as much information as possible.

Entirely unsupervised. All three parts share app embeddings and are trained with a multi-objective joint loss.

User behavior representation

Retention $x_u \in \{0, 1\}^M$ – *what is installed right now*. Multi-hot vector: $x_{um} = 1$ iff app $m = 1, \dots, M$ is on the user's u phone.

Installation sequences – *what was recently added*. Two parallel arrays of length I (latest I ops): $[a_{u,1}^n, \dots, a_{u,I}^n]$ – app IDs, $[d_{u,1}^n, \dots, d_{u,I}^n]$ – dates.

Uninstallation sequences – *what was recently removed*. Same format: app IDs $a_{u,i}^l$ and dates $d_{u,i}^l$.

$$S_u = \left\{ [a_{u,1}^n, \dots, a_{u,I}^n], [d_{u,1}^n, \dots, d_{u,I}^n], [a_{u,1}^l, \dots, a_{u,I}^l], [d_{u,1}^l, \dots, d_{u,I}^l] \right\}$$

Preprocessing rules:

- ≤ 10 ops per user happened in the latest T time intervals
- Drop ubiquitous pre-installed apps (WeChat, Alipay)
- Drop apps pre-installed by manufacturers
- Drop niche apps below a global install threshold

Retention Autoencoder

Purpose: encode high-dimensional sparse retention vector $x_u \in \{0, 1\}^M$ into a dense representation usable by the transformer.

Structure: 3 hidden layers, LeakyReLU activation, final layer uses sigmoid:

$$x^{(p)} = f^{(p)}(x^{(p-1)}\mathbf{W}^{(p)} + b^{(p)}), \quad p \in \{1, 2, 3, 4\}$$

Input $x^{(0)}$ is ℓ_2 -normalized retention x_u .

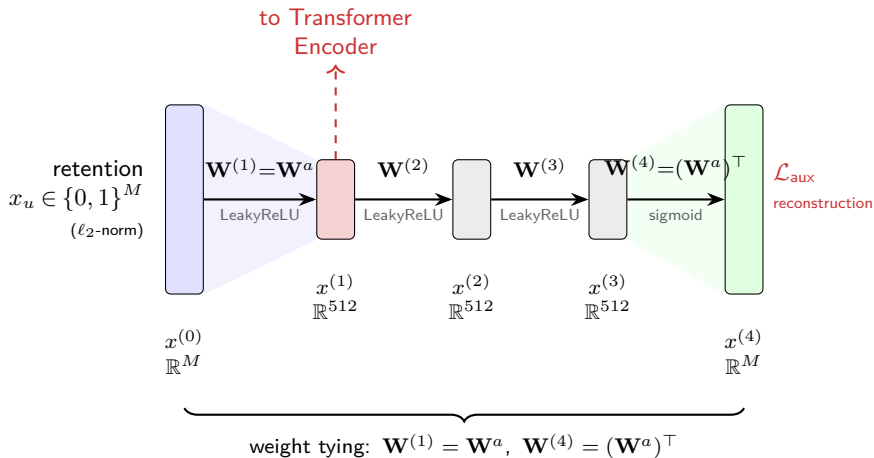
Weight tying: first and last layers share the app embedding matrix $\mathbf{W}^{(1)} = \mathbf{W}^a$:

- Reduces free parameters
- Directly learns meaningful app embeddings from co-occurrence
- Speeds up convergence

Two roles of the autoencoder

1. Learns shared *app embeddings* $\mathbf{W}^a$ from co-occurrence in retention.
2. Produces $x^{(1)} \in \mathbb{R}^{d_{\text{model}}}$, a low-dimensional dense encoding of long-term retention, which seeds the transformer encoder and helps compress long-term interests into the bottleneck.

Retention Autoencoder: architecture



User behavior representation: app embeddings

Problem: $M \sim 10,000$ apps; many long-tail apps appear only hundreds of times among millions of users. A plain ID-lookup embedding learns nothing for rare apps.

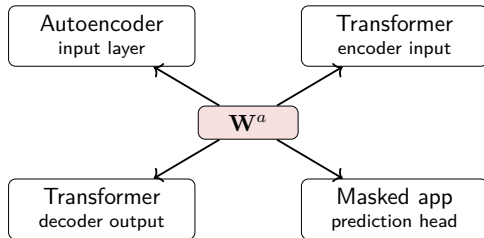
Solution: represent each app m as the *sum* of two learned vectors:

$$\mathbf{w}_m = \underbrace{\mathbf{w}_m^a}_{\text{app ID row}} + \underbrace{\mathbf{w}_m^c}_{\text{category row}} \in \mathbb{R}^{d_{\text{model}}}$$

- $\mathbf{W}^a \in \mathbb{R}^{M \times d_{\text{model}}}$ – app-level lookup (sparse updates)
- $\mathbf{W}^c \in \mathbb{R}^{C \times d_{\text{model}}}$ – category-level lookup ($C \ll M$, dense updates)

If app m is rare, $\mathbf{w}_m^a$ is noisy but $\mathbf{w}_m^c$ is well-trained from all apps in the same category.

Weight tying – $\mathbf{W}^a$ is *shared* across four locations:



Weight tying: (1) reduces parameters, (2) all gradient flows update *one* matrix so even rare app rows receive gradients from reconstruction and masked prediction losses, (3) semantically links the input and output spaces.

Transformer Encoder: embedding layer

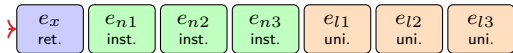
The encoder receives **three signal types** as a unified sequence of token embeddings: Each input token e_i is the **sum** of three components:

$$e_i = \underbrace{\mathbf{w}_{m_i}^a + \mathbf{w}_{m_i}^c}_{\text{app embedding}} + \underbrace{\mathbf{w}_{t_i}^d}_{\text{date embedding}} + \underbrace{\mathbf{w}^{\tau_i}}_{\text{behavior type}}$$

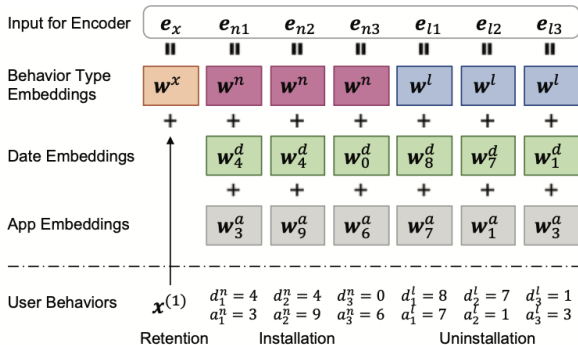
- **App embedding:** app ID + category (shared $\mathbf{W}^a$)
- **Date embedding** $\mathbf{W}^d \in \mathbb{R}^{T \times d_{\text{model}}}$: captures *when* the event happened – crucial for irregular time distributions
- **Behavior type** $\mathbf{w}^\tau \in \{\mathbf{w}^n, \mathbf{w}^w, \mathbf{w}^i\}$: distinguishes retention / installation / uninstallation

Transformer Encoder: input sequence

Sequence fed to encoder (left $\rightarrow$ right):



The retention token $e_x = x^{(1)}$ comes from the autoencoder. Date embeddings replace positional encodings, making the model robust to irregular time gaps.



Transformer encoder: contextual representations

Input: sequence $[e_1, \dots, e_n]$, each $e_i \in \mathbb{R}^{d_{\text{model}}}$

Output: sequence $[h_1, \dots, h_n]$, same shape — each h_i encodes token i *in the context of all other tokens*.

A plain embedding e_i always encodes the same thing regardless of what is around it. A *contextualised* h_i adapts e.g.:

- Installing a gaming app carries different meaning if the user already has 10 games (likely a gamer) vs. none (possibly curious)
- Uninstalling a social app right after installing another social app suggests switching, not exit
- All these interactions are captured *simultaneously*, not chained step by step

How does the transformer compute context? Via **self-attention**: each token scores every other token by relevance, then blends their information proportionally.

Self-attention: how tokens influence each other

For each token i , three linear projections into **query, key, value**:

$$q_i = e_i W^Q, \quad k_j = e_j W^K, \quad v_j = e_j W^V$$

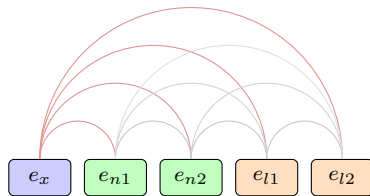
Attention weight – how relevant is token j for token i ?

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d}} \right)$$

Output for token i : $h_i = \sum_j \alpha_{ij} v_j$.

h_i is a learned weighted average of all value vectors. The model adjusts W^Q , W^K , W^V so that each token attends to the most informative context.

Every token can attend to every other
(red = connections involving e_x):



Multi-head attention

Multi-head attention runs H heads *in parallel*, each with its own $W_h^Q, W_h^K, W_h^V \in \mathbb{R}^{d \times d/H}$:

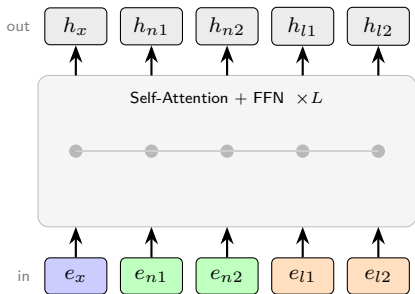
$$\text{head}_h = \text{Attention}(EW_h^Q, EW_h^K, EW_h^V)$$

$$\text{MultiHead}(E) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O$$

$W^O \in \mathbb{R}^{d \times d}$ mixes across all heads, restoring $\mathbb{R}^{n \times d}$.

Heads are architecturally identical – diversity emerges from random initialisation and independent gradient updates.

Transformer encoder: structure



Each encoder **block** (repeated L times):

1. **Multi-Head Self-Attention** — every token attends to all others; H parallel heads capture different aspects
2. **Add & Normalize** — residual connection + layer norm
3. **Feed-Forward Network** — two-layer MLP applied independently at each position
4. **Add & Normalize** — residual connection + layer norm

In AETN: $L = 2$ encoder layers, $H = 8$ attention heads, $d_{model} = 512$

Modified multi-head self-attention

Motivation: retention, bottleneck, installation, and uninstallation are *heterogeneous*. Each install/uninstall event is one operation; retention and bottleneck summarize many operations. Standard self-attention treats them equally.

Modification: concatenate the retention (or bottleneck) vector to *every key and value* in each attention head:

1. In the **encoder**: retention representation is appended to K and V
2. In the **decoder**: user embedding (bottleneck) is appended to K and V

Effect: every attention step directly sees the retention/user context, not just through positional proximity.

Why it helps

- Encoder fuses installation/uninstallation with retention more efficiently
- Decoder can reconstruct sequences using both the user embedding and the long-term retention signal
- AUC gain in next-week install prediction: **+0.0069** vs. vanilla

Transformer Decoder & Bottleneck

Bottleneck layer produces user embedding $\tilde{e} \in \mathbb{R}^{128}$:

- Single hidden autoencoder on top of encoder's final hidden state
- Activation: $\tanh$
- Dimension: $d_{\text{model}} = 512 \rightarrow d_{\text{emb}} = 128$

Retention reconstruction from bottleneck:

- Multi-layer perceptron + sigmoid
- Loss: sigmoid cross-entropy ($\mathcal{L}_{\text{aux}}$)

Decoder reconstructs install/uninstall sequences:

- *Non-autoregressive* (parallel decoding)
- Input queries: date + behavior type embeddings
- Modified attention: bottleneck appended to K , V
- Output softmax weight $\mathbf{W}^\Phi \in \mathbb{R}^{d_{\text{model}} \times M}$ shared with encoder (weight tying)

Full weight tying

Multi-objective joint training

Three tasks trained simultaneously with a shared loss $\mathcal{L} = \mathcal{L}_{\text{main}} + \mathcal{L}_{\text{aux}} + \mathcal{L}_{\text{mask}} + \mathcal{L}_{\text{reg}}$:

Task 1 — Main Reconstruction $\mathcal{L}_{\text{main}}$

Reconstruct:

- retention (from bottleneck via MLP)
- install/uninstall sequences (from decoder)

Loss: sigmoid / softmax cross-entropy.

Indispensable core task.

Task 2 — Auxiliary Retention $\mathcal{L}_{\text{aux}}$

Separately reconstruct retention in the autoencoder part.

Stabilizes transformer training, improves convergence speed.

AUC gain in install prediction: **+0.0023**.

Task 3 — Masked App Prediction $\mathcal{L}_{\text{mask}}$

BERT-style: randomly mask apps in install/uninstall sequences, predict masked app IDs.

Acts as data augmentation.

AUC gain: **+0.0115** across all install categories.

AETN: key dimensions

Component	Dimension
Retention input	$x_u \in \{0, 1\}^{10\,000}$
Autoencoder encoding	$x^{(1)} \in \mathbb{R}^{512}$
Token embedding	$e_i \in \mathbb{R}^{512}$
Sequence length	$n = 1 + 2 \times 10 = 21$
Encoder input	$E \in \mathbb{R}^{21 \times 512}$
Per-head projection (W_h^Q, W_h^K, W_h^V)	$\mathbb{R}^{512 \times 64}$ ($H=8$ heads)
Single head output	$\mathbb{R}^{21 \times 64}$
After concat	$\mathbb{R}^{21 \times 512}$
W^O	$\mathbb{R}^{512 \times 512}$
FFN	$512 \rightarrow 1024 \rightarrow 512$

Experiments & Results

Offline evaluation: three downstream tasks

Tasks (AUC metric):

- **T1:** Next-week installation prediction (4 niche app categories)
- **T2:** Look-alike audience extension (out-of-vocabulary app)
- **T3:** Feed recommendation (cross-domain, Tencent Wi-Fi Manager)

Baselines:

- **DAE** — denoising autoencoder on retention only
- **AETN w/o $\mathcal{L}_{\text{mask}}$** — no masked app prediction
- **AETN w/o $\mathcal{L}_{\text{aux}}$** — no auxiliary retention task
- **V-AETN** — vanilla (unmodified)

Model	T1 Avg AUC	T2 AUC	T3 AUC
DAE	0.7392	0.8175	0.6358
w/o $\mathcal{L}_{\text{mask}}$	0.7908	0.8290	0.6395
w/o $\mathcal{L}_{\text{aux}}$	0.8000	0.8301	0.6403
V-AETN	0.7954	0.8307	0.6401
AETN	0.8023	0.8309	0.6406

- AETN beats DAE by **+0.0631 AUC** avg on T1 — significant in industry where 0.1% matters
- Every ablation degrades quality — all three tasks contribute

Online A/B testing results

Production deployment at Tencent Mobile Manager (Feb 2020, 9 days), split by user ID.

• PV CTR and Engagement					improve most — AETN captures short-term interests from install/uninstall that DAE misses	
Tab	Model	UV CTR	PV CTR	Engagement	Clicks	
Good Morning	+DAE	+1.26%	+2.23%	+2.27%	+2.11%	
	+AETN	+2.81%	+4.79%	+5.24%	+3.86%	
Find	+DAE	+4.12%	+2.29%	+2.82%	+2.11%	• Find tab gains more: users browse all day (more exposure), making personalization more impactful
	+AETN	+4.15%	+5.96%	+8.14%	+3.23%	

- Model trained in ~60 hours on 4 × Tesla M40 GPUs on tens of millions of users

Embeddings updated daily for active users (feature updating strategy — cheaper than full model retraining).

Summary

What AETN does:

- Unsupervised user embeddings from retention + install + uninstall
- Encoder-decoder with information bottleneck ($d = 128$)
- Date embeddings handle irregular time distributions
- Shared app embeddings + category IDs handle long-tail sparsity
- Weight tying couples autoencoder and transformer tightly
- Modified self-attention injects retention/user context at every step
- Three-task joint training (reconstruction

Lessons for general embedding design:

- Model heterogeneous signals *together*, not separately
- Use time embeddings instead of position encodings when events are irregular
- Sparsity $\rightarrow$ tie embeddings + add coarser-grained features (category)
- BERT-style masking works beyond text: applies to any discrete sequence
- Multi-task training: auxiliary tasks improve both quality and convergence speed